Session 16 · Tuesday 20 October 2026 · Posted after class. Every trace here is computed by posb.logic, not typed.

Ticks, glitches and depth — answers

Item 1 is worked in full on the handout.

2 — Compile XNOR

XNOR is 1 when A and B agree, so it is the OR of the two agreeing rows: $(\neg A \wedge \neg B) \vee (A \wedge B)$. Each row becomes a NOR of the complements of its literals; the two row terms are then **NOR**ed together and that result inverted — NOR-then-invert, because NOR is the gate you have and OR is not.

GENE	IS	COMPUTES
$\neg A$	NOT A	the complement of A
$\neg B$	NOT B	the complement of B
P	NOR(A, B)	both low
Q	NOR$(\neg A, \neg B)$	both high
R	NOR(P, Q)	XOR
out	NOT R	XNOR

Six genes. Longest path 4, shortest path 3. Count them: $A \to P \to R \to$ out is three genes; $A \to \neg A \to Q \to R \to$ out is four.

The number people reach for is **3 and 2**, and it comes from forgetting the output gene — counting the rows of the table above rather than the genes on a route.

A different group's network is not worse for being different. An exhaustive search over networks of two-input NOR gates says the smallest XNOR is **four** gates:

$$P = \mathrm{NOR}(A, B), \quad S = \mathrm{NOR}(A, P), \quad T = \mathrm{NOR}(B, P), \quad \mathrm{out} = \mathrm{NOR}(S, T).$$

Two fewer genes to clone and one less protein lifetime of delay, for the same truth table. The table does not choose between them; a compiler does, and that step is "logic synthesis" in the Cello pipeline.

Its paths really are 3 and 2 — $A \to P \to S \to$ out, and $A \to S \to$ out. So if you built the four-gate network and wrote 3 and 2, you are right. The error above is 3 and 2 reported for the *six-gene* network. Same two numbers, different circuits.

What both have in common, and it is the point of the session: **the paths are not the same length.** Hold that thought until item 5, where it turns out not to be the thing that matters either.

3 — The other circuit

$R_1 = \text{NOT}\, A$, $R_2 = \text{NOR}(B, R_1)$, $\text{out} = \text{NOT}\, R_2$.

t	R_1	R_2	OUT		t	R_1	R_2	OUT
$A=0,\ B=0$					$A=0,\ B=1$			
0	0	0	0		0	0	0	0
1	1	1	**1**		1	1	0	1
2	1	0	**0**		2	1	0	1
3	1	0	**1**		3	1	0	1
4	1	0	1		4	1	0	1
$A=1,\ B=0$					$A=1,\ B=1$			
0	0	0	0		0	0	0	0
1	0	1	**1**		1	0	0	1
2	0	1	**0**		2	0	0	1
3	0	1	0		3	0	0	1
4	0	1	0		4	0	0	1

(a) It settles to

A	B	out
0	0	1
0	1	1
1	0	0
1	1	1

which is $\text{NOT}(A \wedge \neg B)$ — the complement of item 1's classifier.

(b)

CASE	GLITCHES?	WHICH KIND
$A=0,\ B=0$	yes	**dynamic** — 0, 1, 0, 1, 1: supposed to change once, changed three times
$A=0,\ B=1$	no	clean
$A=1,\ B=0$	yes	**static-0** — 0, 1, 0, 0, 0: supposed to stay at 0
$A=1,\ B=1$	no	clean

Two of the four. Worth noticing: a circuit can be wrong in some input cases and right in others, so testing the one case you happen to try tells you almost nothing about the rest.

(c) In all four tables the output is **1 at $t = 1$**, whatever the inputs are. The gene that decided it is the output gene itself: it is a NOT, and a NOT with no repressor present is transcribing. At $t = 0$ there is no protein in the cell at all, so it comes on, and it comes on before anything has had time to tell it not to.

Note the qualifier. It is **not** true that every NOT and NOR in the circuit is on at $t = 1$ — look at R_1 in the $A = 1$ tables and R_2 in the $B = 1$ tables, both 0. A gene reading an **input** reads the real value immediately. It is a gene reading only **other genes** that is wrong before it is right, and the output here is one of those.

(d) Settled at $A = B = 1$ the state is $R_1 = 0$, $R_2 = 0$, out $= 1$. Drop both inputs:

t	R_1	R_2	OUT
0	0	0	**1**
1	1	1	**1**
2	1	0	**0**
3	1	0	**1**
4	1	0	1

The settled answer is 1 in both input cases, and the output dips to 0 on the way: a **static-1** hazard.

It could not have appeared in part (b), and neither could it anywhere else on this sheet. From an empty cell every gene starts at 0, so the output's first move is always *upward* — it can hold 0 and blip (static-0), or end at 1 having wobbled (dynamic), but it can never hold 1 and dip, because it did not start at 1. **A static-1 needs a circuit that is already running.**

That is also the honest footnote on the vocabulary: "static-0", "static-1" and "dynamic" come from digital design, where they describe what happens when a settled circuit has an input changed. Most of this session is about something else — a circuit being born — and in that regime the *kind* of glitch is not independent information at all. Settles to 0 and it is a static-0; settles to 1 and it is dynamic. Every time.

4 — The four-gene chain

$$t_{1/2}(4) \;\approx\; \left(4 - \tfrac{1}{3}\right)\tau = \mathbf{3.67}\,\tau$$

and against the three-gene chain, **exactly $1\,\tau$ later** — the rule of thumb is linear in n, so the difference is 1 by construction. That is the number to carry out of the session: *one more gene, one more protein lifetime.*

The exact values, if you want to check the rule rather than use it, are $t_{1/2}(3) = 2.674\,\tau$ and $t_{1/2}(4) = 3.672\,\tau$, a difference of $0.998\,\tau$. The rule is good to 0.2% here.

Why people underestimate it. The first stage is the fast one: $t_{1/2}(1) = \ln 2 = 0.693\,\tau$, well under a lifetime. Every stage after it costs very nearly a full τ (0.985, 0.996, 0.998, …), so a chain feels cheap when you think about its first gene and is not.

And this is the floor. Hooshangi's measured cascades are far slower: circuit 1 settles by about 120 min and circuit 3, two stages deeper, not until about 600 (p. 3583). Two things their system has that ours does not.

Their stages are **ultrasensitive**, so a small upstream change is nearly invisible downstream until it is large; and the repressors are expressed far above the level needed for full repression, which they verify experimentally, so the next stage does not move until enough protein has been made or diluted away to cross a **threshold** (p. 3585). Our τ is a free parameter and theirs is minutes of real protein turnover, so read the comparison as a shape, not as a bound.

5 — What actually decides it

Ticking (ii) and (iii) at $A = 0$, $B = 0$:

t	(II) out	(III) out
0	0	0
1	0	0
2	**1**	0
3	0	0

(ii) is fail-safe **and** balanced and still blips. (iii) is fail-safe, unbalanced — A reaches the output in two genes and B in one — and clean in all four cases, and it computes $A \wedge \neg B$, which is item 1's classifier. So:

CIRCUIT	OFF WHEN EMPTY?	PATHS EQUAL?	RESULT
item 1's classifier	no	no	glitches 4/4
$Z = \mathrm{NOR}(\neg A, \neg B)$	no	yes	glitches 3/4
(i) padded with an activator	yes	no	glitches 4/4
(ii) that, with equal legs too	yes	yes	glitches 3/4
(iii) two genes	yes	**no**	**clean**

Read the second column and then the third. Neither one tracks the result. "OFF when empty" is wrong on two rows; "paths equal" is wrong on three.

(a) What is true of (iii) and false of the others: **nothing it depends on is a repressor that arrives late.** In (iii) the output promoter has two signals, and the one that arrives late — g, at depth 2 — sits at the **activating** site, while B, which arrives at once, sits at the repressing site. In (i) and (ii) the late signal is a repressor, or is produced by a gene that was itself wrong.

(b) The rule:

> ***An absent activator holds a gene off. An absent repressor lets it fire.*** *So a gene is safe if it sits one step from the inputs, and therefore is never wrong — or if its promoter requires an activator and no repressor it depends on arrives later than that activator.*

Two things to notice about the shape of that. It is an **inequality**: a repressor arriving *early* is harmless, which is exactly why (iii) survives being unbalanced. And it has to hold at **every** gene on the way to the output, not just the last one — (ii) fails it because Z is a NOR, so Z is already wrong, and the BUF on the end relays that wrongness faithfully one tick later. You cannot repair a circuit at its output.

A NOT or a NOR has no activating site at all, so it fires the instant the cell exists. That is why a circuit built only from them can never be made clean by changing the last promoter, and why the repair in class needed a **hybrid** gene — activation required, repression dominant.

This is a *sufficient* condition, not a characterisation: there are clean circuits it refuses to certify. It is checked by exhaustion over every two- and three-gate network in this model (`GateNetwork.cold_start_safe`, and `tests/test_logic.py`), and the model gives every gate the same unit delay, which real promoters do not.